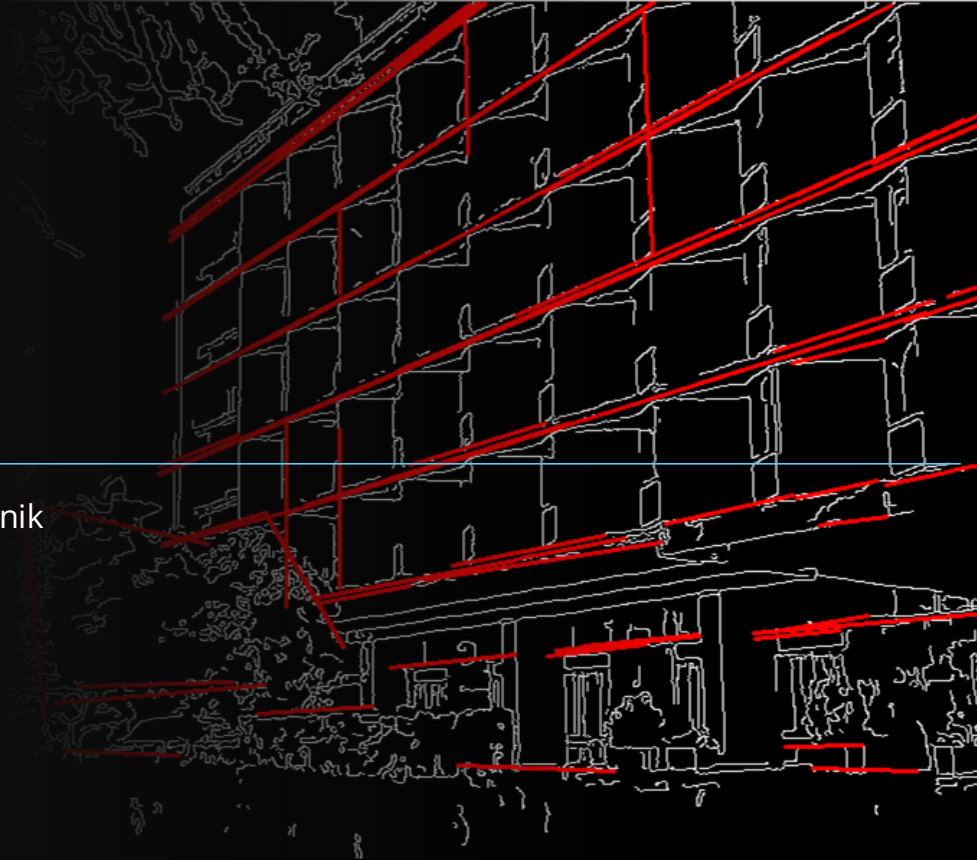


19/8/2015

Fitting

Slides are from Svetlana Lazebnik



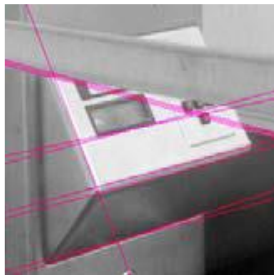
Fitting

- We've learned how to detect edges, corners, blobs. Now what?
- We would like to form a higher-level, more compact representation of the features in the image by grouping multiple features according to a simple model



Fitting

Choose a *parametric model* to represent a set of features



simple model: lines



simple model: circles



complicated model: car

Fitting: Challenges

Case study: Line detection



- **Noise** in the measured feature locations
- **Extraneous data:** clutter (outliers), multiple lines
- **Missing data:** occlusions

Fitting: Overview

- Least squares line fitting
- Robust fitting
- RANSAC
- Hough transform
- Generalized Hough transform

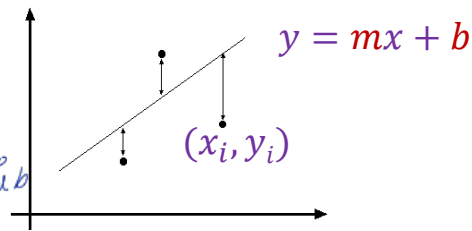
Least Squares Line fitting: First attempt

- Data: $(x_1, y_1), \dots, (x_n, y_n) \Rightarrow$ *measured points (true)*
- Line equation: $y_i = mx_i + b \Rightarrow$ *prediction.*
- Find (m, b) to minimize

Reduce TSS

$$E = \sum_{i=1}^n (y_i - mx_i - b)^2$$

differentiate to find min m and b



- Equivalent to finding least-squares solution to: *finding WLS*

$$\begin{matrix} \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} & \begin{pmatrix} m \\ b \end{pmatrix} & = & \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \\ X & B & & Y \end{matrix}$$

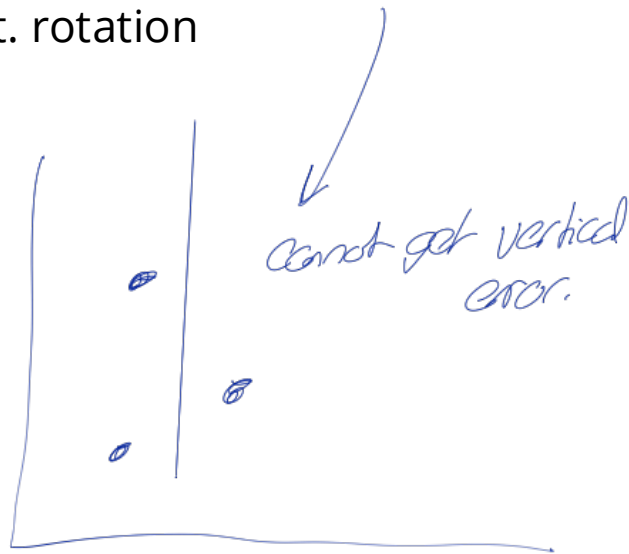
- Solution is given by $X^T X B = X^T Y$

$$B = (X^T X)^{-1} X^T Y$$

$$XB = Y \Rightarrow B = \underbrace{(X^T X)^{-1} X^T}_{\text{pseudo inverse}} Y$$

Is this a good solution? *Nope*

- Slope-intercept parametrization fails for vertical lines
- Solution is not equivariant w.r.t. rotation

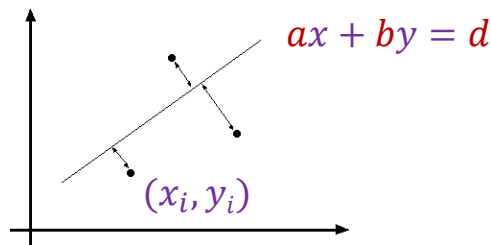


So we use TLS

Total least squares

- Line parametrization: $ax + by = d$
 - (a, b) is the *unit normal* to the line (i.e., $a^2 + b^2 = 1$) $\begin{bmatrix} a \\ b \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = d$ *normal vector to the line*
 - d is the distance between the line and the origin

can work with vertical lines



- Perpendicular distance between point (x_i, y_i) and line $ax + by = d$ (assuming $a^2 + b^2 = 1$):

$$|ax_i + by_i - d| \quad \text{distance}$$

- Objective function:

$$E = \sum_{i=1}^n (ax_i + by_i - d)^2$$

Total Least Squares

$$E = \sum_{i=1}^n (ax_i + by_i - d)^2$$

- Solve for d first:

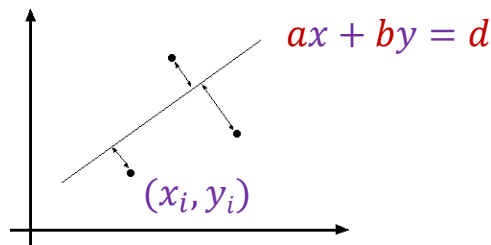
$$\frac{\partial E}{\partial d} = -2 \sum_{i=1}^n (ax_i + by_i - d) = 0$$

need to sum
 d , n times
we get nd

$$d = \frac{a}{n} \sum_{i=1}^n x_i + \frac{b}{n} \sum_{i=1}^n y_i = a\bar{x} + b\bar{y}$$

get min E
wrt d

$$\Rightarrow d = a\bar{x} + b\bar{y}$$

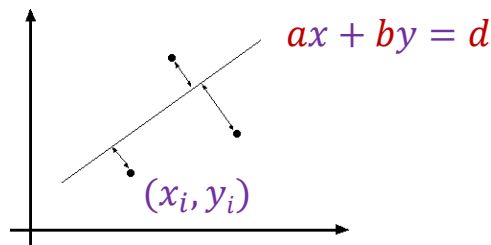


Total least squares

$$E = \sum_{i=1}^n (ax_i + by_i - d)^2$$

- Solve for d first: $d = a\bar{x} + b\bar{y}$
- Plugging back in:

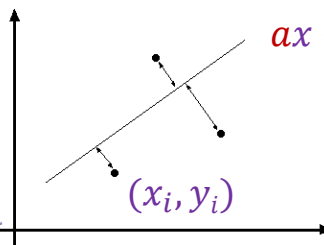
$$E = \sum_{i=1}^n (a(x_i - \bar{x}) + b(y_i - \bar{y}))^2$$



$$ax + by + c = 0 \Rightarrow y = -\frac{a}{b}x - \frac{c}{b}$$

$$\text{for 2 points } y = \frac{(y_2 - y_1)}{(x_2 - x_1)}x - \frac{c}{b}$$
$$= -\frac{(y_2 - y_1)}{(x_1 - x_2)}x - \frac{c}{b}$$

Total Least Squares

$$E = \sum_{i=1}^n (ax_i + by_i - d)^2$$


$ax + by = d$

(x_i, y_i)

- Solve for d first: $d = a\bar{x} + b\bar{y}$
- Plugging back in:

$$E = \sum_{i=1}^n (a(x_i - \bar{x}) + b(y_i - \bar{y}))^2 = \left\| \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix} \begin{pmatrix} a \\ b \end{pmatrix} \right\|^2$$

$U \quad N$

min is 0

x_i and y_i are vectors so we consider x is a col and y is a row and call X

$x = \text{np.hstack}(x, y)$

when $a=b=0$ we get the min but it is a trivial sol not useful. So we keep $a^2 + b^2 = 1$

- We want to find N that minimizes $\|UN\|^2$ subject to $\|N\|^2 = 1$
 - Solution is given by the eigenvector of $U^T U$ associated with the smallest eigenvalue

*Just memorize this.
can we svd for this (singular value decomposition)*

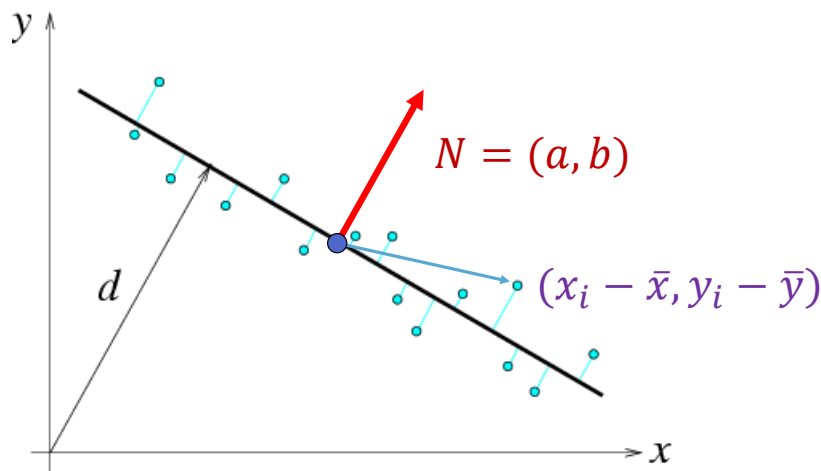
Total Least Squares

Leads at singular value decomposition

$$U = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix}_{n \times 2}$$

$$U^T U = \begin{bmatrix} \sum_{i=1}^n (x_i - \bar{x})^2 & \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \\ \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) & \sum_{i=1}^n (y_i - \bar{y})^2 \end{bmatrix}_{2 \times 2}$$

second moment matrix



SVD tells how the line is spread out

SVD gives

data point
matrix

$$\begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ \vdots & \vdots \end{bmatrix}_{m \times n} = U_{m \times m} \cdot S_{m \times n} \cdot V^T_{n \times n}$$

$V^T \Rightarrow$ direction of variance of data

$V^T[0] \Rightarrow$ Direction of max variance

$V^T[-1]/V^T[1] \Rightarrow$ perpendicular direction

$S \Rightarrow$ how much variance in each direction

$U \Rightarrow$ how to get each data point (rotate/stretch)
or how data points relate to the directions

so $ax + by + c = 0$

$$V^T[-1] = a, b$$

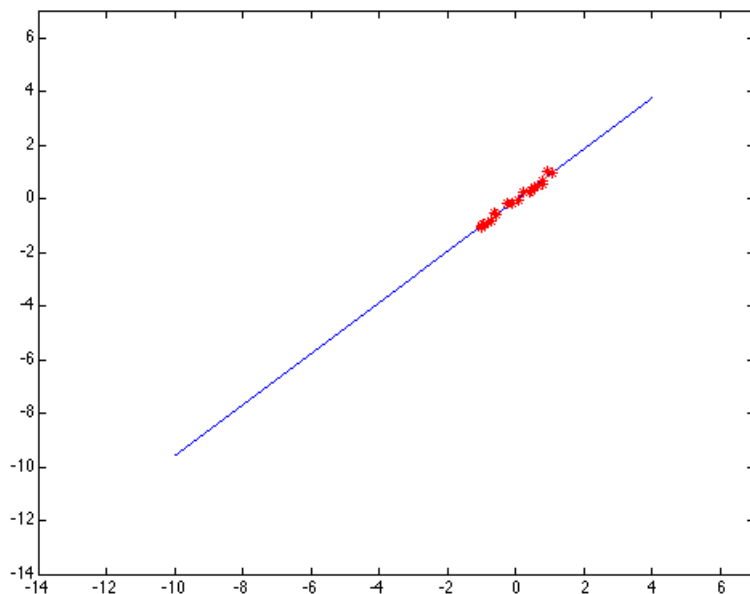
* then get $\bar{x}$ and $\bar{y}$ for the
inliers and

$$a\bar{x} + b\bar{y} + c = 0 \quad \text{to find } c$$

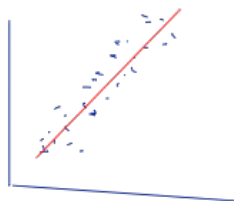
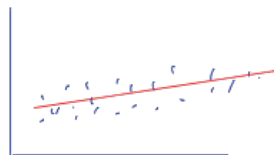
Do not use any data point. Use the
centroid.

Least Squares: Robustness to Noise

Least squares fit to the red points:



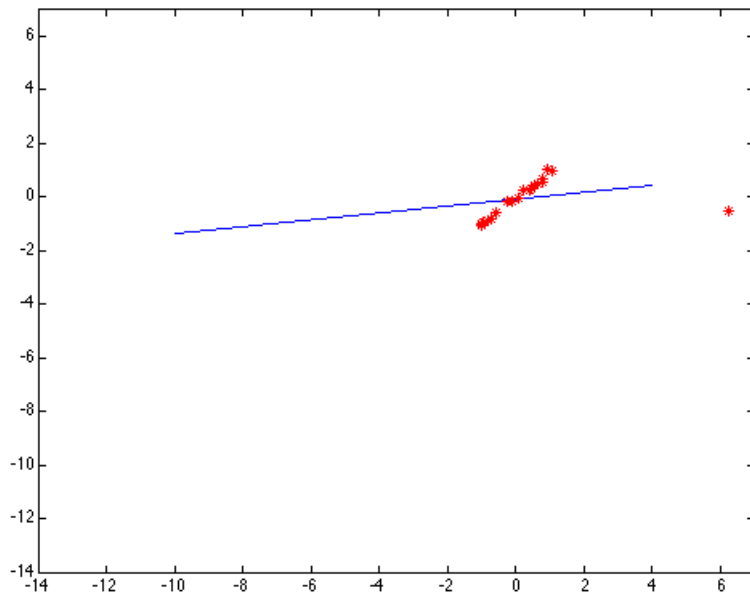
*TLS is rotational
invariant
OLS method is
not.*



*same TLS
different OLS*

Least Squares: Robustness to noise

- Least squares fit with an outlier:



Problem: squared error heavily penalizes outliers

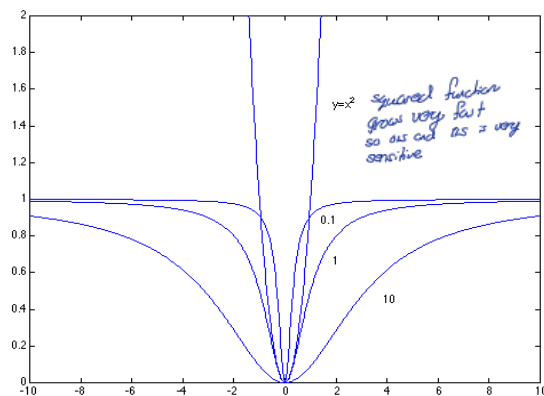
Robust Estimators

- General approach: find model parameters θ that minimize

$$\sum_i \rho_{\sigma}(r(x_i; \theta))$$

$r(x_i; \theta)$: residual of x_i w.r.t. model parameters θ

ρ_{σ} : robust function with scale parameter σ , e.g., $\rho_{\sigma}(u) = \frac{u^2}{\sigma^2 + u^2}$



Robust estimators

- General approach: find model parameters θ that minimize

$$\sum_i \rho_\sigma(r(x_i; \theta))$$

$u \Rightarrow y_i - \hat{y}_i$ in our
 $|ax_i - by_i - d|$ in TLS

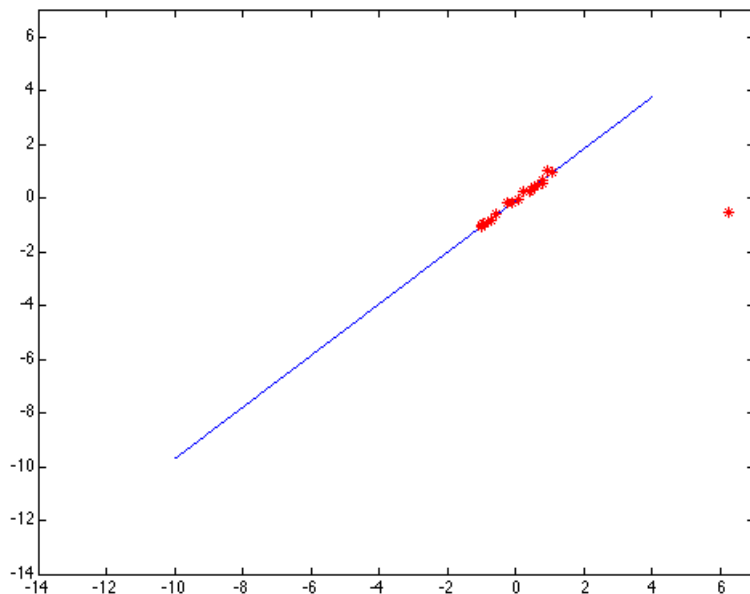
$r(x_i; \theta)$: residual of x_i w.r.t. model parameters θ

This can stop giving
large errors after
a certain point

ρ_σ : robust function with scale parameter σ , e.g., $\rho_\sigma(u) = \frac{u^2}{\sigma^2 + u^2}$

- Nonlinear optimization problem that must be solved iteratively
 - Least squares solution can be used for initialization
 - Scale of robust function should be chosen carefully

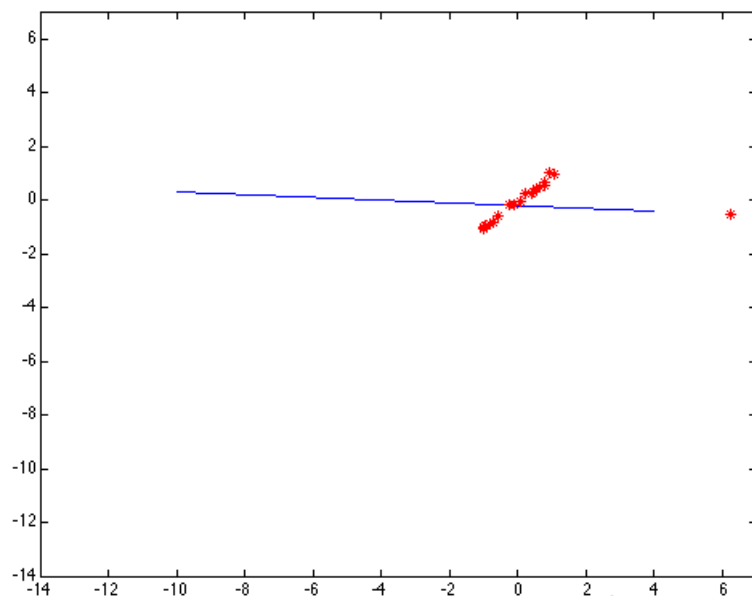
Choosing the Scale: Just Right



The effect of the outlier is minimized

Choosing the Scale: Too Small

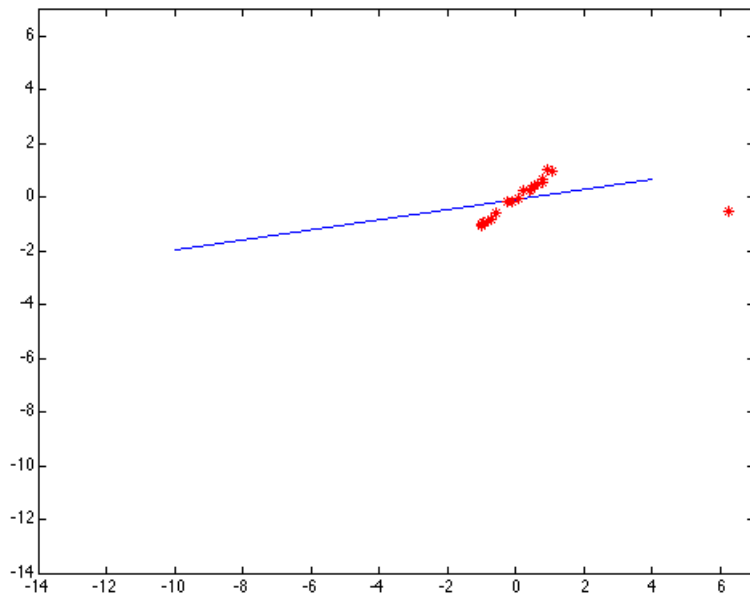
too small



The error value is almost the same for every point and the fit is very poor

Choosing the Scale: Too large

σ too large.



Behaves much the same as least squares

when σ^2 is too large it is like ϵ^2/σ^2 which is a scaled version of ϵ^2 (LS)

Fitting: Overview

- Least squares line fitting
- Robust fitting
- RANSAC

conserves set of inliers.

we get a sample and ask who agrees and get the average of those who agree.

if no one agrees, that is an outlier and we can remove.



→ 30 agree → average



→ no one agrees → discard.

for a line we need 2 samples and both must be non-outliers
for a circle all 3 samples must be non-outliers. high prob of contamination.

Summary

A line needs 2 points.

So select 2 points and draw a line and get dist to all other data points.

We decide how much we misclassify depending on a threshold. (can stop early if majority are outliers)

Do this for several iterations given by

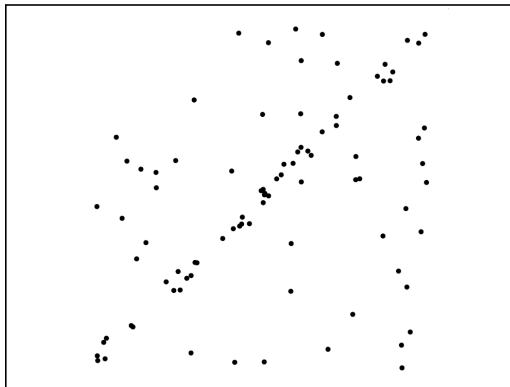
$$N = \frac{\log(1-p)}{\log[1-(1-c)^s]}$$

get the line with highest votes.

Finally if we need, we can extract the inliers from that model and use TLS to get a better fitting line.

Voting Schemes

- Robust fitting can deal with a few outliers – what if we have very many?
- Basic idea: let each point *vote* for all the models that are compatible with it
 - Hopefully the outliers will not vote consistently for any single model
 - The model that receives the most votes is the best fit to the image



RANSAC

- Random sample consensus: very general framework for model fitting in the presence of outliers
- Outline:
 - Randomly choose a small initial subset of points
 - Fit a model to that subset
 - Find all inlier points that are “close” to the model and reject the rest as outliers
 - Do this many times and choose the model with the most inliers

M. Fischler and R. Bolles. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Comm. of the ACM, Vol 24, pp 381-395, 1981

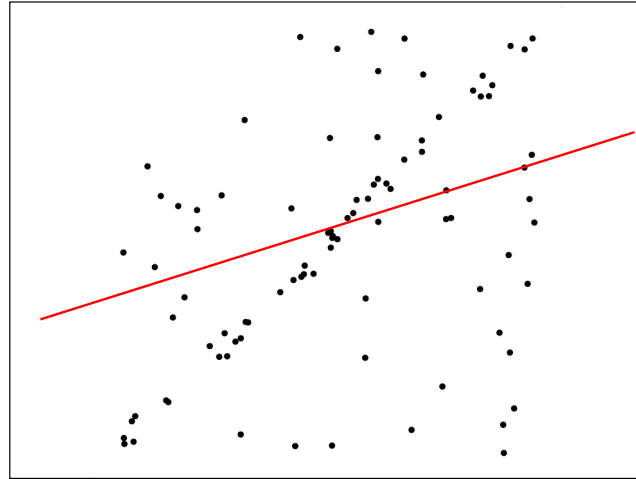
Source: Lazebnik

RANSAC for Line Fitting Example



Source: R. Raguram

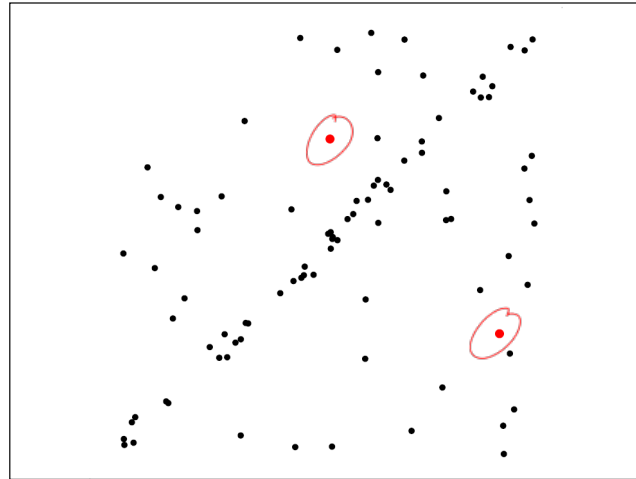
RANSAC for Line Fitting Example



Least-squares fit

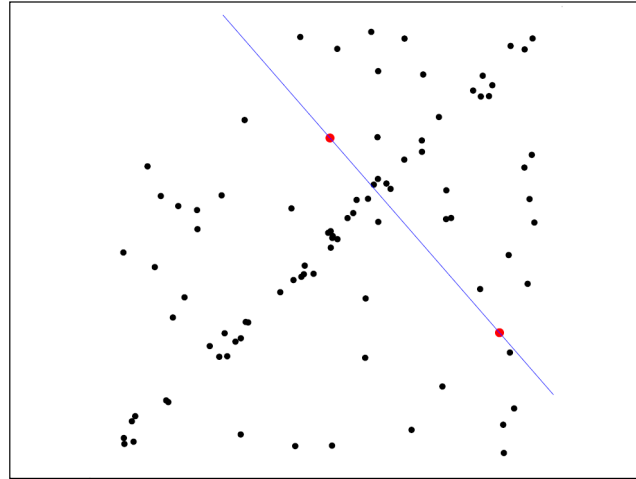
Source: R. Raguram

RANSAC for Line Fitting Example



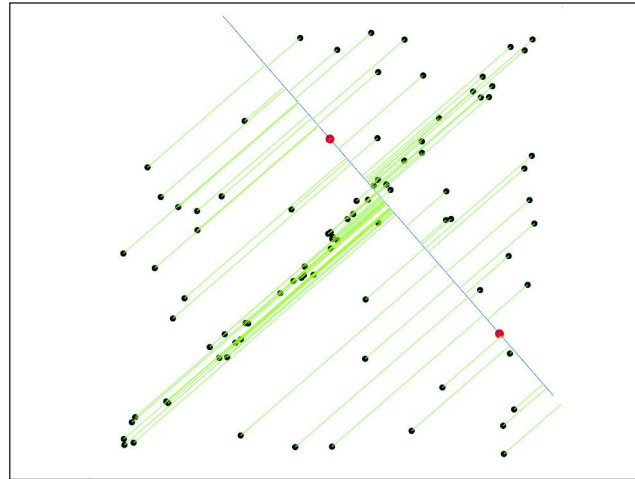
1. Randomly select minimal subset of points

RANSAC for Line Fitting Example



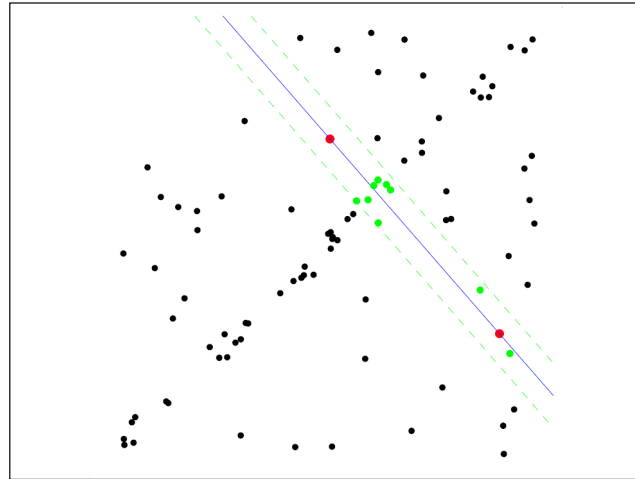
1. Randomly select minimal subset of points
2. Hypothesize a model

RANSAC for Line Fitting Example



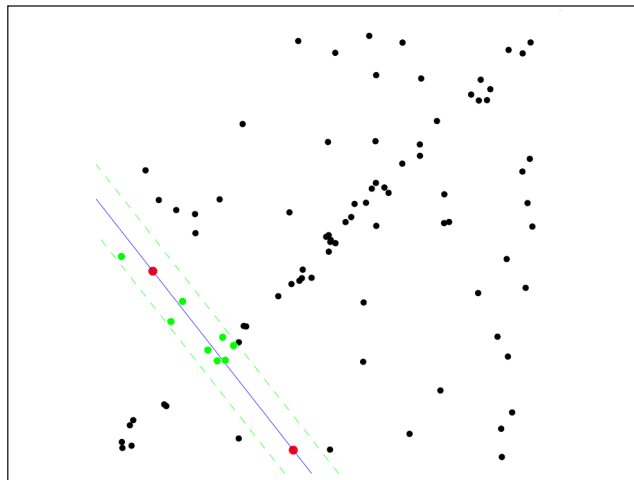
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function

RANSAC for Line Fitting Example



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model

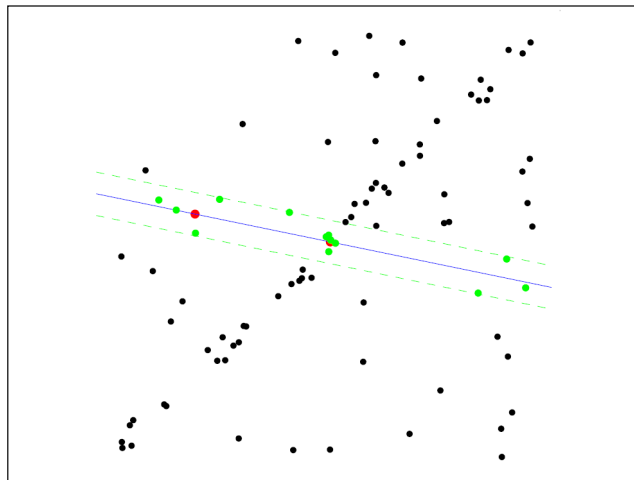
RANSAC for Line Fitting Example



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

Source: R. Raguram

RANSAC for Line Fitting Example

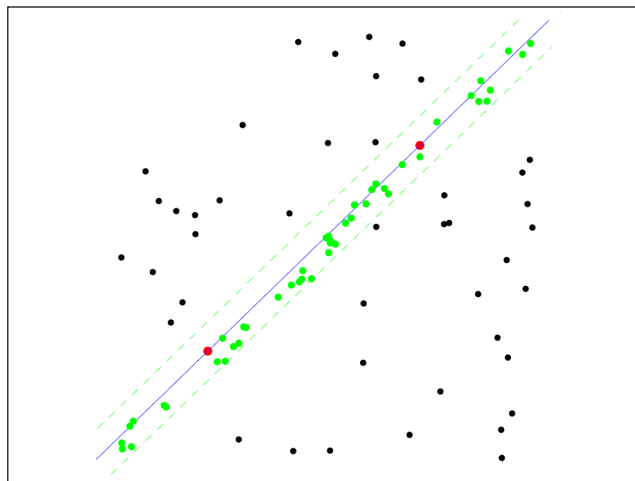


1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

RANSAC for Line Fitting Example

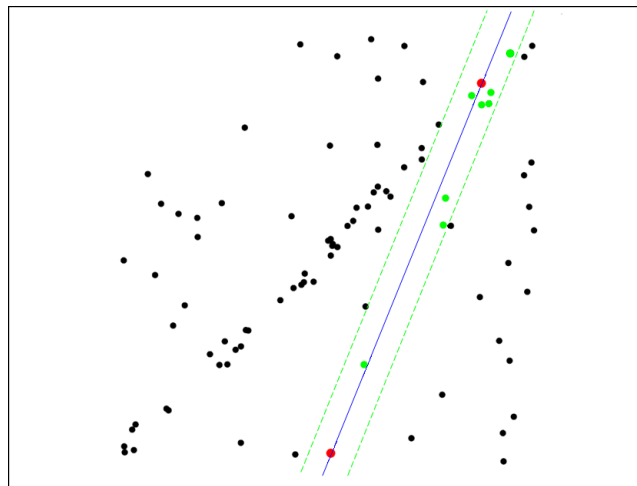
*How do we select the threshold
That is the hyper parameter.*

Uncontaminated sample



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify loop*

RANSAC for Line Fitting Example



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

RANSAC: Choosing the Parameters

Repeat N times:

- Draw s points uniformly at random
- Fit model to these s points
- Find *inliers* to the model among the remaining points (points whose distance or residual w.r.t. model is less than t)
- If there are d or more inliers, accept the model and refit using all inliers

$s=2$ line
 $s=3$ circle

hyper param?

how many points must be inside.

In normal RANSAC we run it and choose the model with highest votes.

But now if inlier $> d$ we accept that and use TLS to fit inlier and calculate error for test. Repeat this for all models with inlier $> d$ and get the model with the lowest score.

RANSAC: Choosing the Parameters

- Initial number of points s
 - Typically minimum number needed to fit the model
- Distance threshold t for inliers
 - Need suitable assumptions, e.g., given zero-mean Gaussian noise with std. dev. σ , $t = 1.96\sigma$ will give ~95% probability of capturing all inliers
- Consensus set size d
 - Should match expected inlier ratio
- Number of samples N
 - Choose N so that, with probability p , at least one random sample is free from outliers (e.g. $p = 0.99$)
 - Assuming an outlier ratio of e :

$$(1 - (1 - e)^s)^N = 1 - p$$
$$N = \log(1 - p) / \log(1 - (1 - e)^s)$$

$$\text{outlier ratio} = e \quad \left[\begin{array}{l} \text{how many} \\ \text{outliers in} \\ \text{population} \end{array} \right]$$

$$\text{inlier ratio} = 1 - e$$

$$\text{prob of } s \text{ points being inliers} = (1 - e)^s$$

$$\text{prob of contamination} \\ \left(\begin{array}{l} \text{at least one sample is an outlier} \\ \text{so we're} \end{array} \right) = 1 - (1 - e)^s$$

All n samples being contaminated

$$[1 - (1 - e)^s]^n$$

let p be the prob of sample not being contaminated

$$[1 - (1 - e)^s]^n = 1 - p$$

set $p = 0.66$ and solve for n to see how many samples we need to do.

(e is a given data)

Formula for the No. of Samples N

Outlier ratio = e (e.g., 40%)

Inlier ratio: $1 - e$

If we have to pick s point (e.g., $s = 2$ for a line)

Probability of all s points being inliers $(1 - e)^s$

Prob. of the sample of s points being contaminated: $1 - (1 - e)^s$

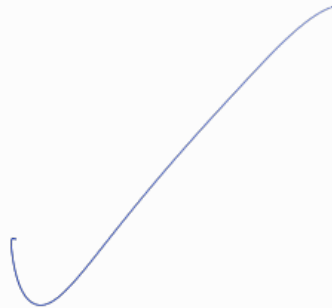
Prob. of all N samplings being contaminated: $[1 - (1 - e)^s]^N$

At least one sample being free of contamination: $1 - [1 - (1 - e)^s]^N$

If we want with probability p , at least one random sample is free from outliers:

$$[1 - (1 - e)^s]^N = 1 - p$$
$$N = \log(1 - p) / \log[1 - (1 - e)^s]$$

take the ceil of N

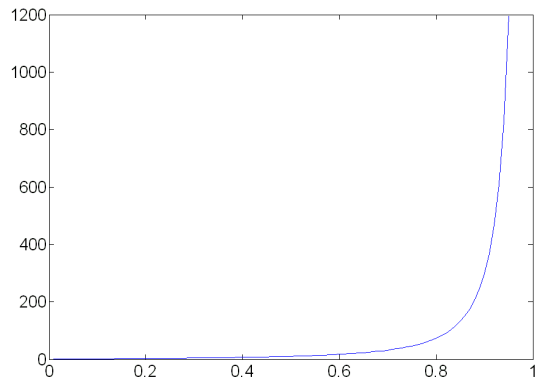


RANSAC: Choosing the Parameters

- Number of samples vs. outlier ratio:

s	proportion of outliers e						
	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

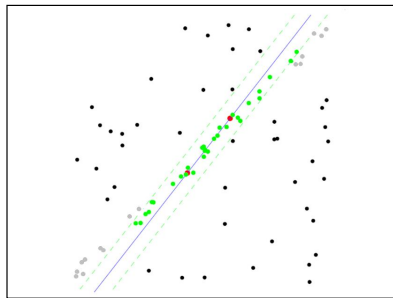
how many sample need to be done in
order to get $p = 0.99$



MLE-SAC is a much
better alt
look at this.

RANSAC pros and cons

- Pros
 - Simple and general
 - Applicable to many different problems
 - Often works well in practice
- Cons
 - Lots of parameters to tune
 - Doesn't work well for low inlier ratios (too many iterations, or can fail completely)
 - Can't always get a good initialization of the model based on the minimum number of samples



Ransac Fitting lines and circles

Lines

Randomly get 2 points (x_1, y_1) (x_2, y_2)
then compute a, b for $ax + by + c = 0$ that comes
from these 2 points

$$\text{set } a = \frac{y_2 - y_1}{x_2 - x_1} \text{ and } b = \frac{x_2 - x_1}{y_2 - y_1}$$

Then compute c

now get distances for all points $|ax + by + c|$

put a threshold and select inliers.

repeat and select the model with most
inliers and take these inliers

now use these to fit a line with TLS.

$$\text{centered matrix} = U S U^T \quad (\text{using svd})$$

$$\text{and get } a, b = V^T [-1]$$

$$\text{now find } c \Rightarrow a\bar{x} + b\bar{y} + c = 0 \quad \text{This is our line}$$

Circle

pick 3 points (x_1, y_1) (x_2, y_2) (x_3, y_3)

equation of a circle

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

substitute the points and get $\textcircled{1} - \textcircled{2}$
 $\textcircled{1} - \textcircled{3}$

to get a matrix equation similar to

$$A \begin{bmatrix} x_c \\ y_c \end{bmatrix} = B$$

Solve it to get x_c and y_c
then use one of the points to get r

Now find inliers.

for all datapoints compute
 $\text{abs}(\sqrt{(x - x_c)^2 + (y - y_c)^2} - r)$ and if
smaller than a threshold it is an inlier.
find the model with the most inliers.

Now find the best circle using the
inliers

For lines we had TLS but not for
circle

So define a loss function and minimize it.

not the same x_c . We used these x_c, y_c, r why for ransac to find the inliers

$$L(c) = \sum_{i=1}^n \left[\sqrt{(x_i - x_c)^2 + (y_i - y_c)^2} - r \right]^2$$

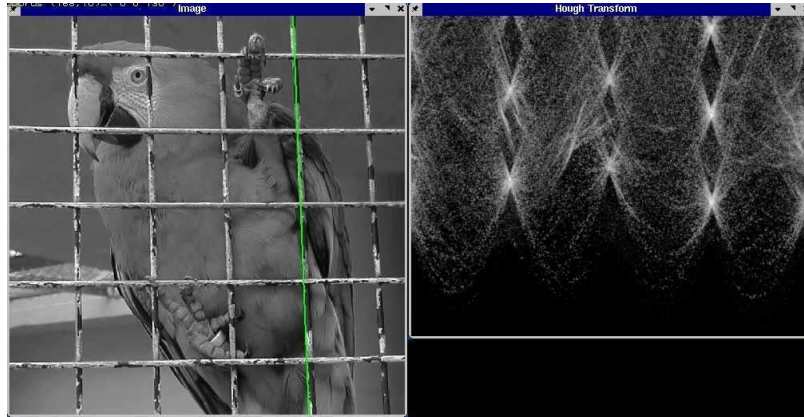
find x_c, y_c, r that minimize this loss for all inliers
That is our model

For the cost function the initial guess is the ransac best-model.

Then it tries to minimize the error by changing x_c, y_c, r

Fitting: Overview

- Least squares line fitting
- Robust fitting
- RANSAC
- Hough transform



Hough Transform

① Doing the prob in a diff space

- Possibly the earliest voting scheme – but still useful!
 - Discretize parameter space into bins
 - For each feature point in the image, put a vote in every bin in the parameter space that could have generated this point
 - Find bins that have the most votes

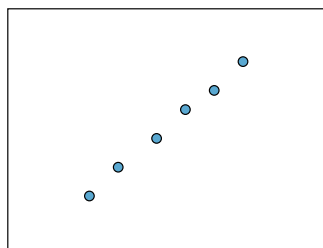
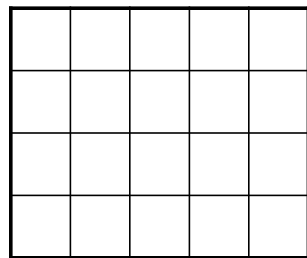
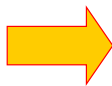


Image space



Hough parameter space

discrete space.

Hough Transform

- What does a **line** in the image space correspond to?
 - A **point** in the parameter space

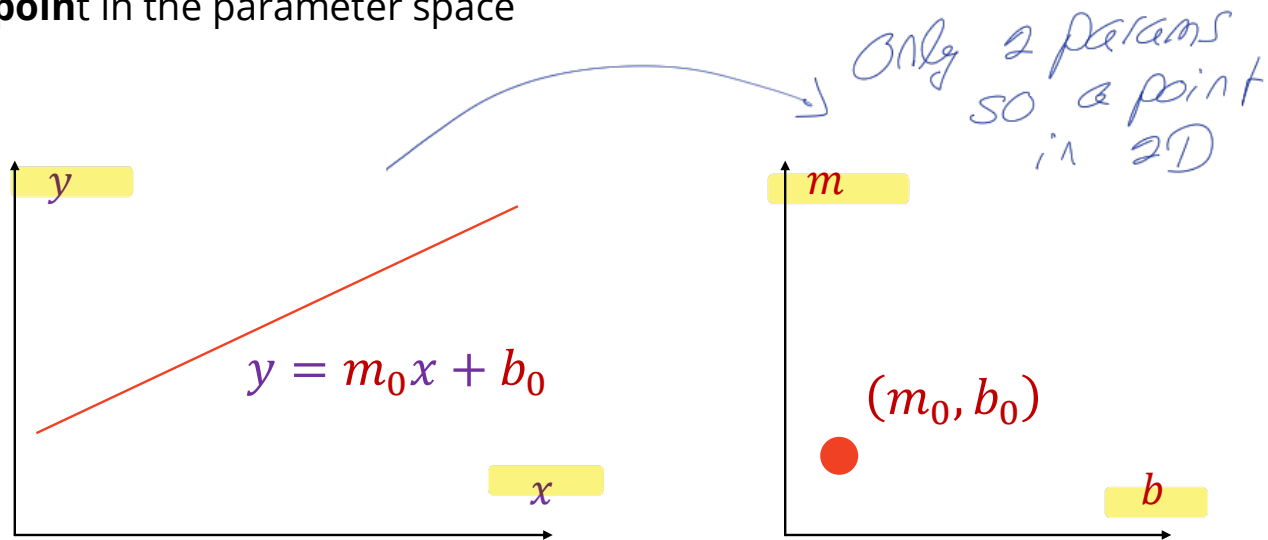


Image space

Parameter space

we don't have access to line. Only a set of points

Hough transform

- What does a **point** in the image space correspond to?
 - A **line** in the parameter space: all (m, b) that satisfy $-b = x_0 m - y_0$

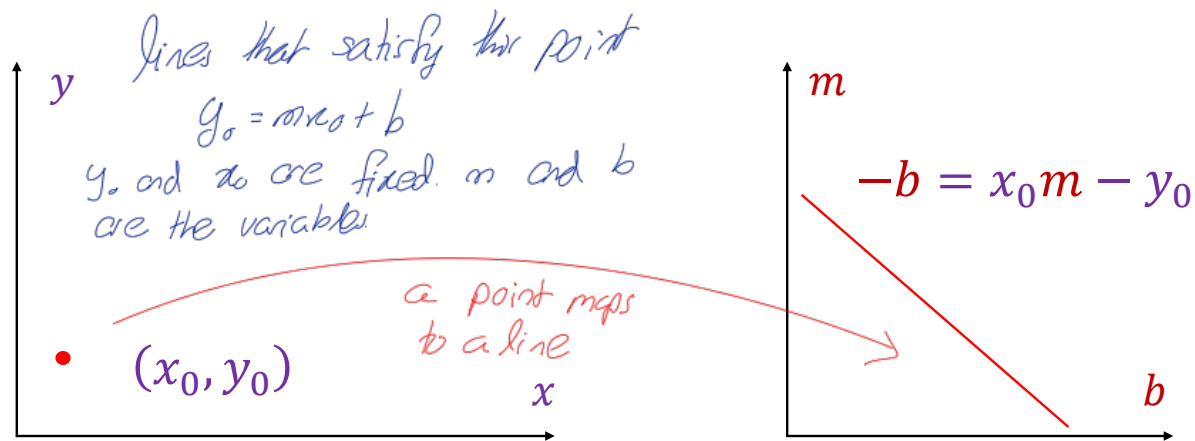


Image space

Parameter space

all lines with these m and b will go through x₀ and y₀

Hough transform

- What about **two points** in the image space?
 - A **point** in the parameter space, corresponding to the unique line that passes through both points

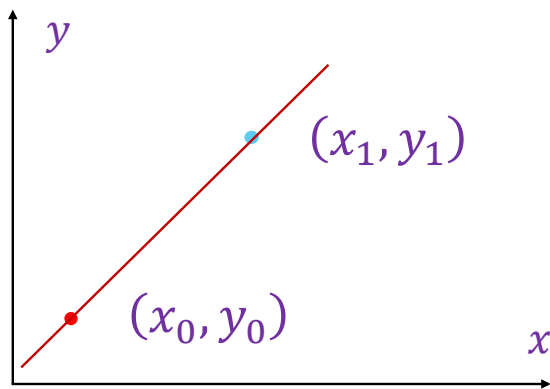
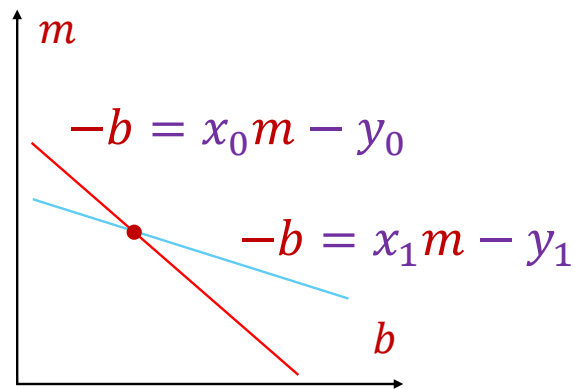


Image space



Parameter space

Hough transform

- What about **many points** in the image space?
 - Plot all the lines in the parameter space and try to find a spot where a large number of them intersect

each point corresponds to a line

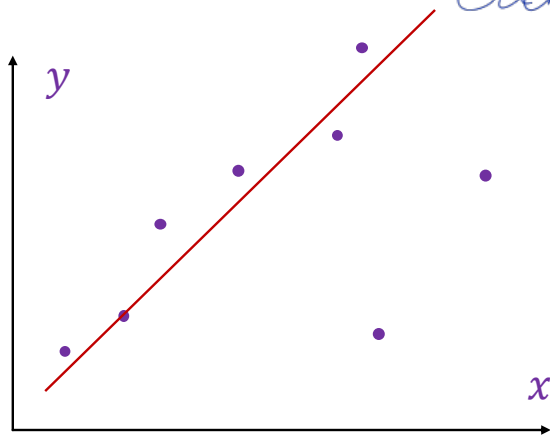
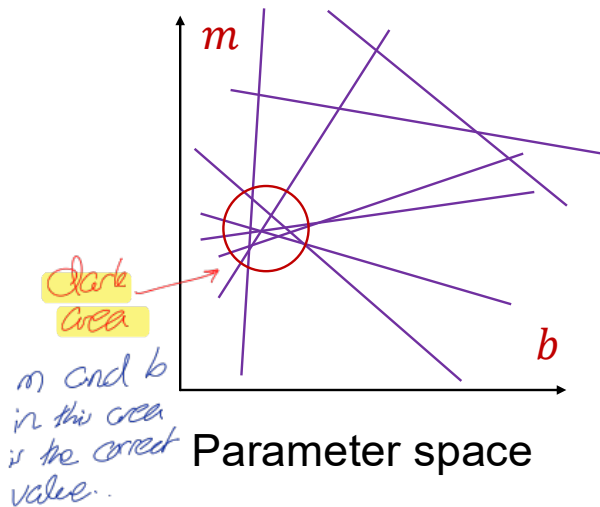


Image space

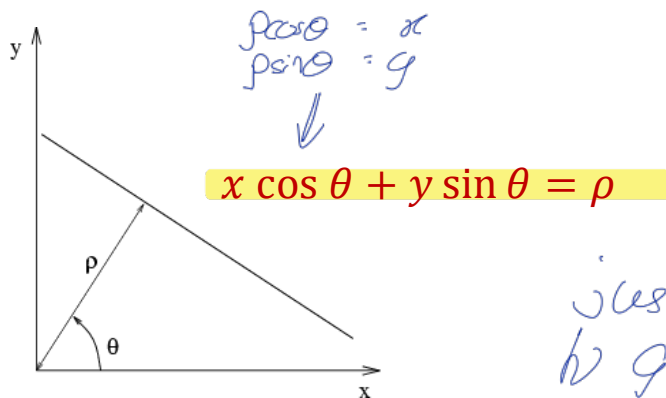


Parameter space

Parameter space representation

- In practice, we don't want to use the (m, b) space!
 - Unbounded parameter domains
 - Vertical lines require infinite m
- Alternative: *polar representation*
 - Each image point (x, y) yields a *sinusoid* in the (θ, ρ) parameter space

instead of (m, b)



$$\begin{aligned} \rho \cos \theta &= x \\ \rho \sin \theta &= y \end{aligned}$$

just another way
to give the line

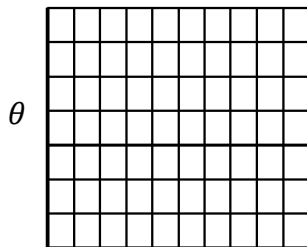
Algorithm outline

→ histogram at 0
no votes yet.

- Initialize accumulator H to all zeros
- For each feature point (x, y)
For $\theta = 0$ to 180

distance — $\rho = x \cos \theta + y \sin \theta$
(max is image diagonal)
-D to +D
 $H(\theta, \rho) += 1$

H: accumulator array (votes)



The voting board
(histogram)

- Find the value(s) of (θ, ρ) where $H(\theta, \rho)$ is a local maximum (perform NMS on the accumulator array)
- The detected line in the image is given by
 $\rho = x \cos \theta + y \sin \theta$

Feature points $\Rightarrow$ a pixel that belongs to an edge (from an edge detector like Canny)

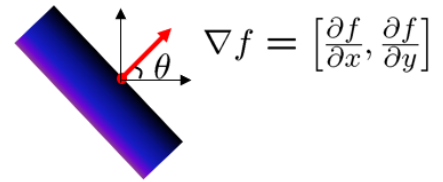
Incorporating Image Gradients

For each edge point (x, y) *angle of normal vector*

θ = gradient orientation at (x, y)

$$\rho = x \cos \theta + y \sin \theta$$

$$H(\theta, \rho) += 1$$



for each point we get a line in H space
similar to $p = y$ and $\theta = \pi$

$$y = a \cos \pi + b \sin \pi$$

this gives every possible line through that point

Now for each point we change $\theta = 0$ to 180
and calculate p and put the (θ, p) pair in
the nearest bin in H . (similar to incrementing each bin in H
the line passes)

so for each point, 180 votes are casted

The Problem: Multiple Peaks for a Single Line

After the voting is complete, your Hough accumulator H doesn't have nice, single-pixel peaks for each line. Instead, you get **blobs** or **clusters of high values**. Here's why:

1. **Quantization Effects:** A perfect line in the image might correspond to $(\theta=44.7^\circ, \rho=100.3)$, but our bins are at $(\theta=45^\circ, \rho=100)$ and $(\theta=44^\circ, \rho=100)$. Votes get scattered across neighboring bins.
2. **Noise in the Image:** Slightly non-straight edges or small errors in edge detection mean that points don't lie perfectly on a mathematical line, so their votes spread around the true peak.
3. **Thick Lines:** If an edge is several pixels thick, you'll get multiple ρ values for roughly the same θ .

The result is that a single, actual line in the image creates a **bright region** in the Hough space, not a single bright pixel.

The Solution: Non-Maximum Suppression (NMS)

NMS is a technique to find the "true" center of these bright blobs and suppress the surrounding, slightly less bright values. The goal is to ensure that **each actual line produces only one detected line** in our final result.

How NMS Works on the Hough Accumulator

Let's walk through the process:

Step 1: Scan the Accumulator

We go through every cell (θ, ρ) in the Hough accumulator H .

Step 2: Check the Neighborhood

For each cell (θ, ρ) , we look at its **local neighborhood** (typically a 3×3 or 5×5 window centered on the cell).

Step 3: Compare with Neighbors

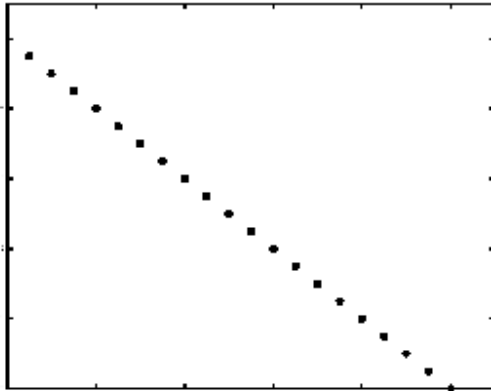
We ask: "Is the value at $H(\theta, \rho)$ **greater than ALL** of its neighbors in this window?"

- If **YES**: Then (θ, ρ) is a **local maximum**. We keep it as a candidate peak.
- If **NO**: Then (θ, ρ) is not a local maximum. We **suppress** it (set it to zero or ignore it).

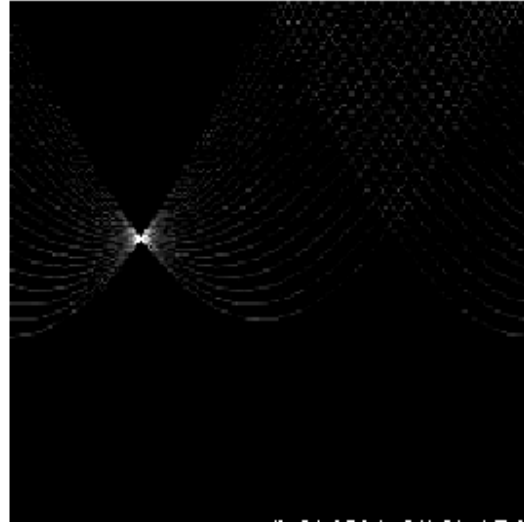
Step 4: Apply a Threshold (Usually)

After finding all local maxima, we typically apply a **vote threshold**. Only peaks with votes above this threshold are considered real lines. This filters out weak detections that might just be from random noise.

Basic Illustration



features

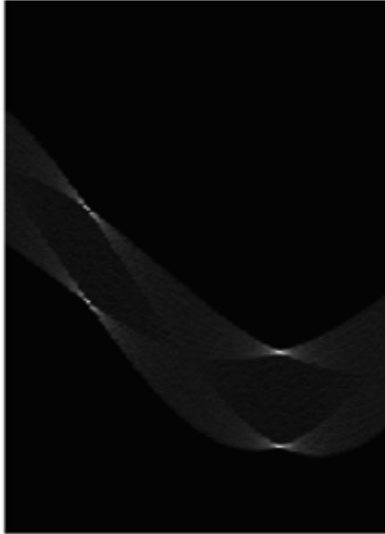


votes

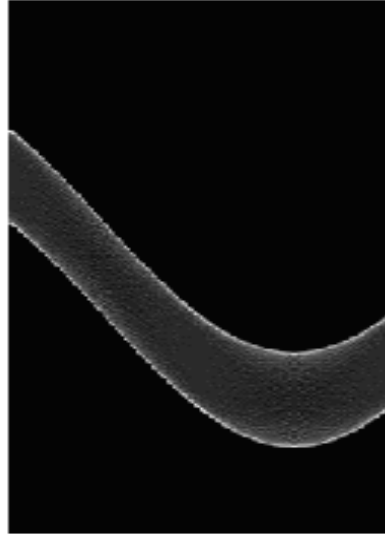
[Hough transform demo](#)

Other Shapes

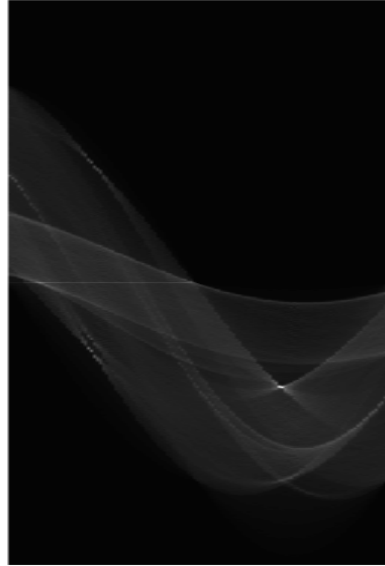
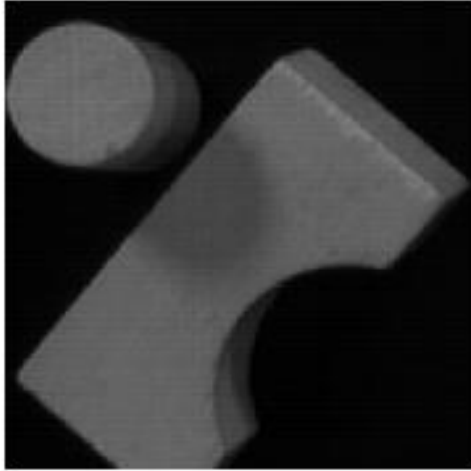
Square



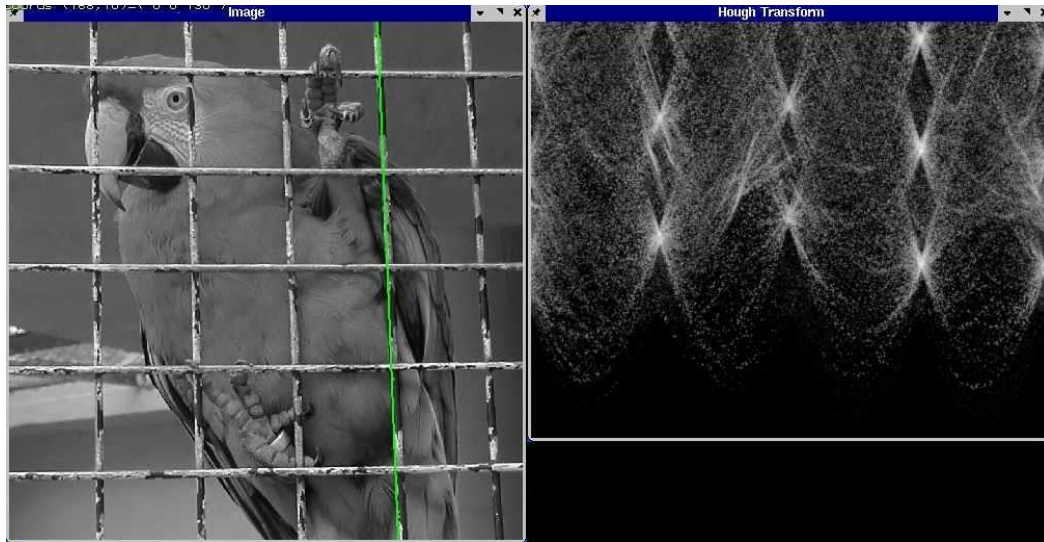
Circle



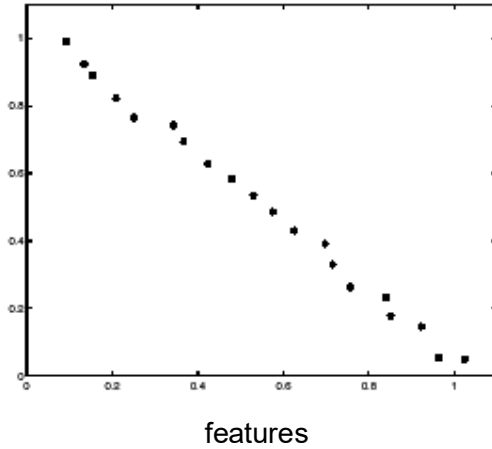
Several Lines



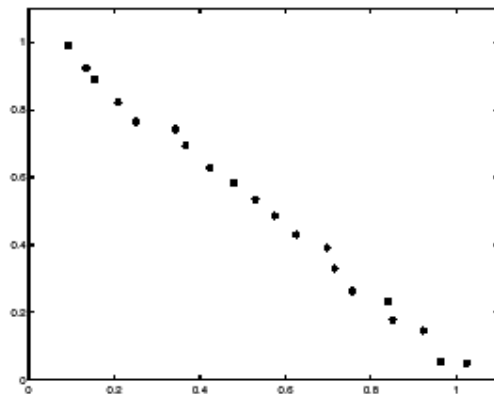
A More Complicated Image



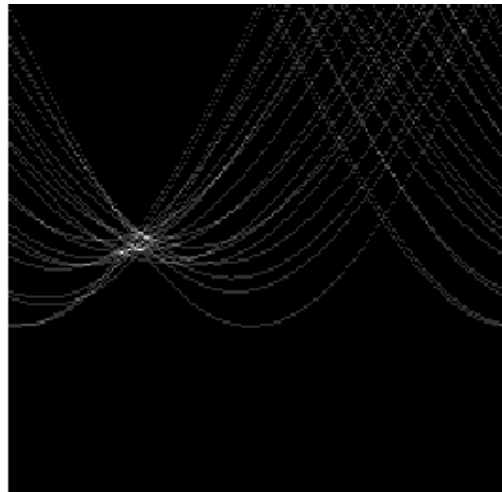
Effect of Noise



Effect of noise



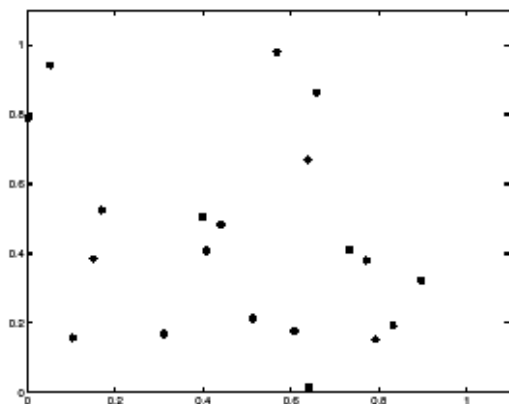
features



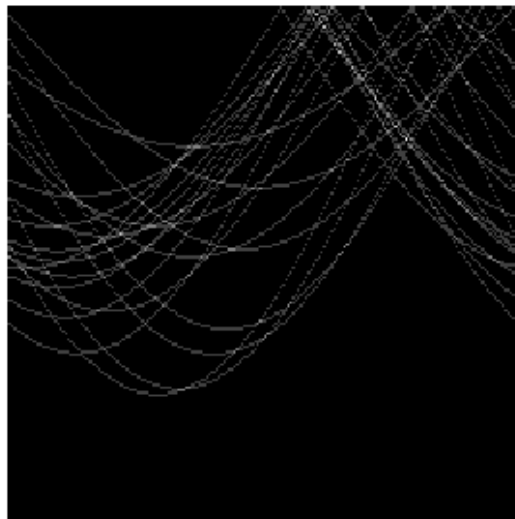
votes

Peak gets fuzzy and hard to locate

Effect of Outliers



features



votes

Uniform noise can lead to spurious peaks in the array

Dealing with Noise

noise affects the grid size.

- How to choose a good grid discretization?
 - **Too coarse:** large votes obtained when too many different lines correspond to a single bucket
 - **Too fine:** miss lines because some points that are not exactly collinear cast votes for different buckets
- Increment neighboring bins (smoothing in accumulator array)
- Try to get rid of irrelevant features
 - E.g., take only edge points with significant gradient magnitude

Hough Transform: Pros and Cons

- Pros
 - Can deal with non-locality and occlusion
 - Can detect multiple instances of a model
 - Some robustness to noise: noise points unlikely to contribute consistently to any single bin
 - Leads to a surprisingly general strategy for object localization (more on this next)
- Cons
 - Complexity increases exponentially with the number of model parameters – in practice, not used beyond three or four dimensions
 - Non-target shapes can produce spurious peaks in parameter space
 - It's hard to pick a good grid size

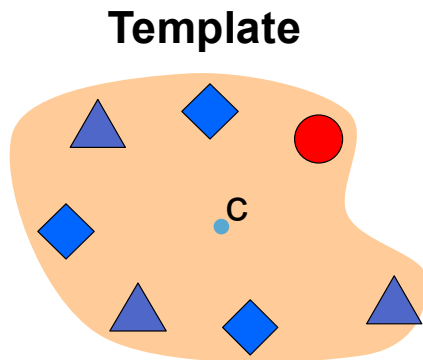
Fitting: Overview

- Least squares line fitting
- Robust fitting
- RANSAC
- Hough transform
- Generalized Hough transform

Can detect arbitrary shapes.

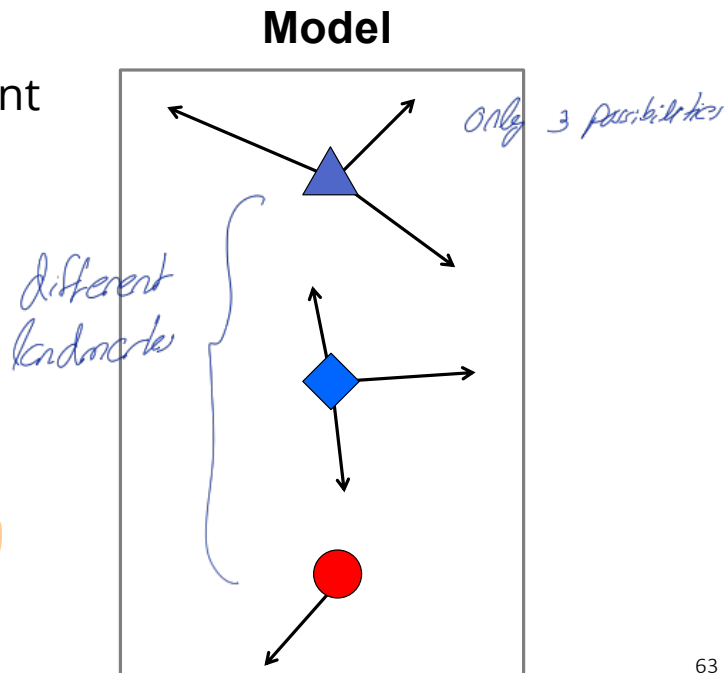
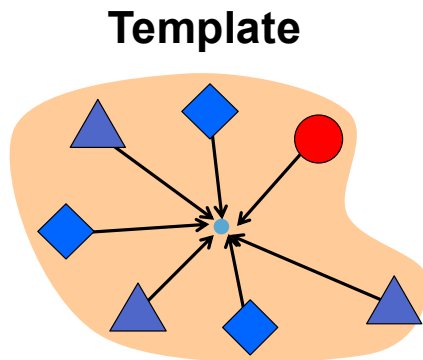
Generalized Hough Transform

- We want to find a template defined by its reference point (center) and several distinct types of landmark points in stable spatial configuration



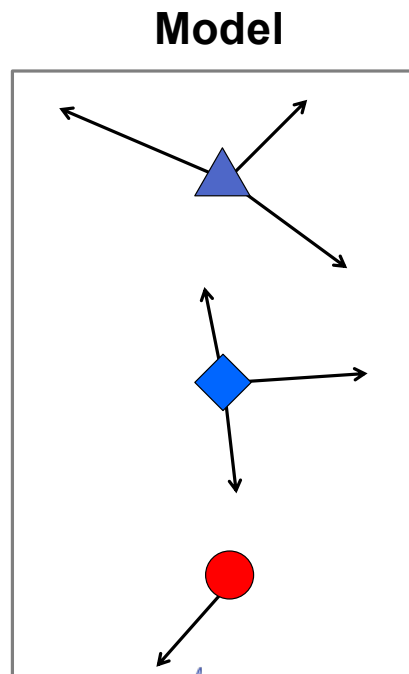
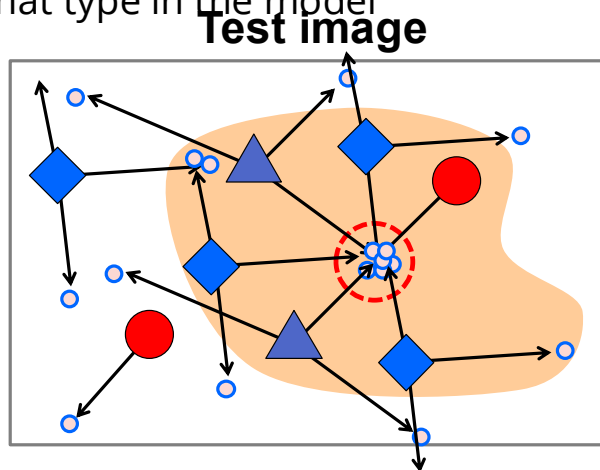
Generalized Hough Transform

- Template representation: for each type of landmark point, store all possible displacement vectors towards the center



Generalized Hough Transform

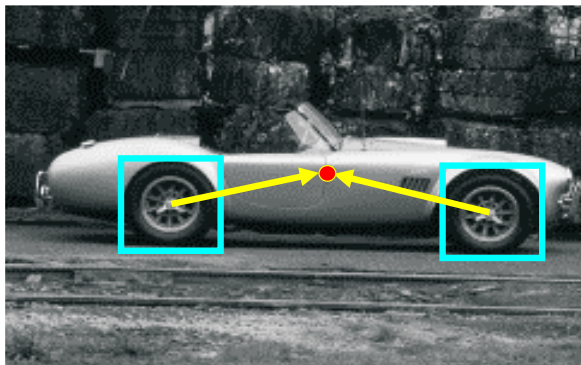
- Detecting the template:
 - For each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model



*not scale invariant
check about rotational invariance*

Application in Recognition

- Index displacements by “visual codeword”



training image



visual codeword with
displacement vectors

B. Leibe, A. Leonardis, and B. Schiele, [Combined Object Categorization and Segmentation with an Implicit Shape Model](#), ECCV Workshop on Statistical Learning in Computer Vision 2004

Application in Recognition

- Index displacements by “visual codeword”



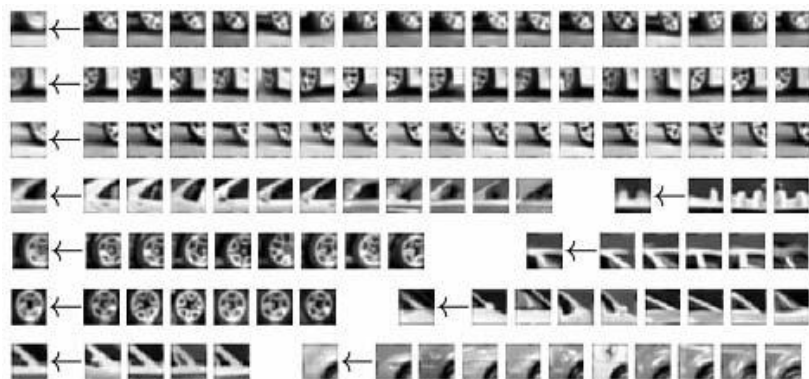
test image

Source: Lazebnik

B. Leibe, A. Leonardis, and B. Schiele, [Combined Object Categorization and Segmentation with an Implicit Shape Model](#), ECCV Workshop on Statistical Learning in Computer Vision 2004

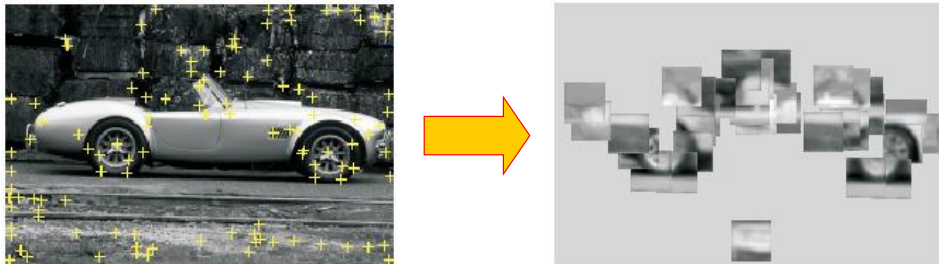
Implicit Shape Models: Training

1. Build *codebook* of patches around extracted interest points using clustering (more on this later in the course)



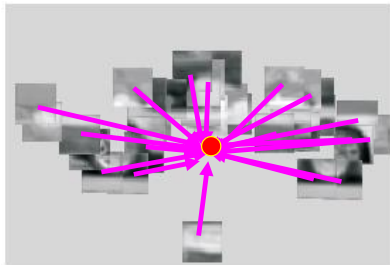
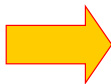
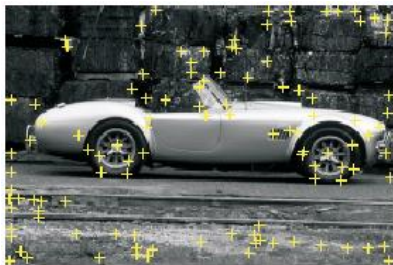
Implicit Shape Models: Training

1. Build *codebook* of patches around extracted interest points using clustering
2. Map the patch around each interest point to closest codebook entry



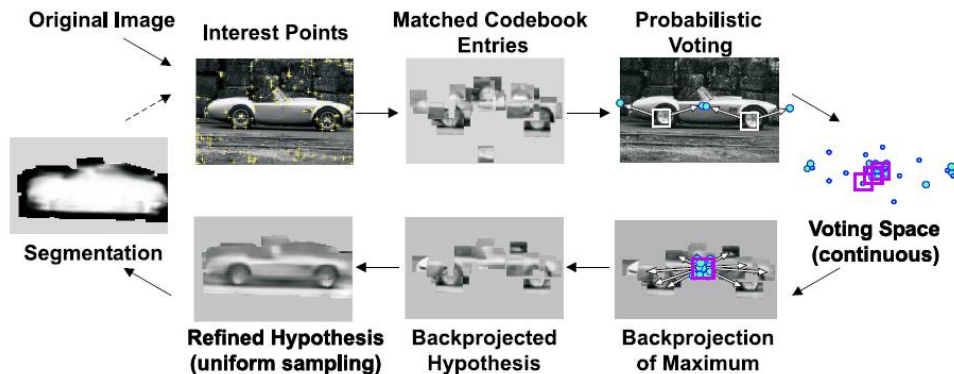
Implicit Shape Models: Training

1. Build *codebook* of patches around extracted interest points using clustering
2. Map the patch around each interest point to closest codebook entry
3. For each codebook entry, store all positions it was found, relative to object center

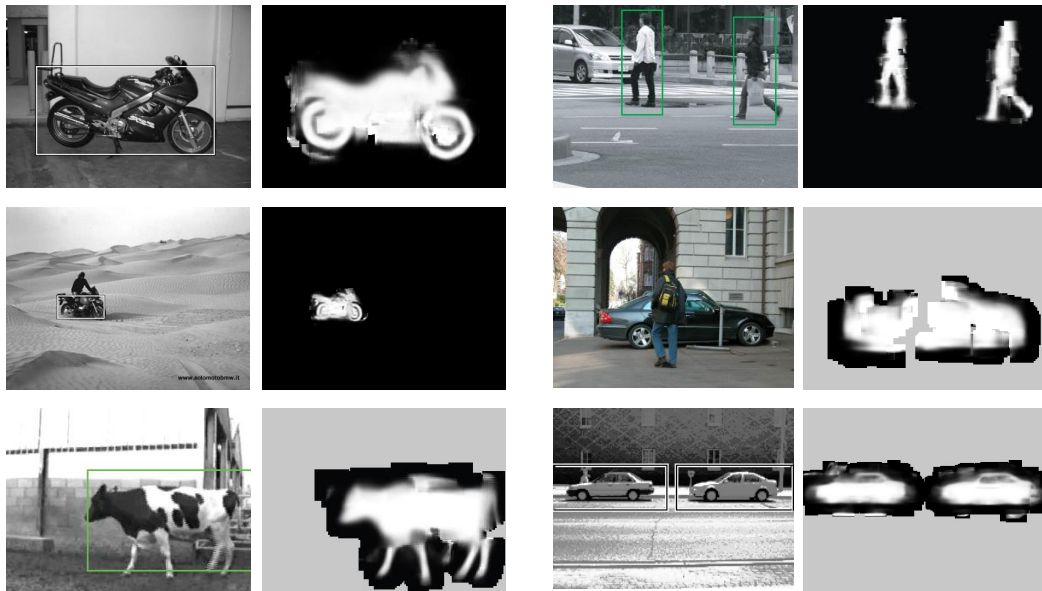


Implicit Shape Models: Training

1. Given test image, extract patches, match to codebook entry
2. Cast votes for possible positions of object center
3. Search for maxima in voting space
4. Extract weighted segmentation mask based on stored masks for the codebook occurrences



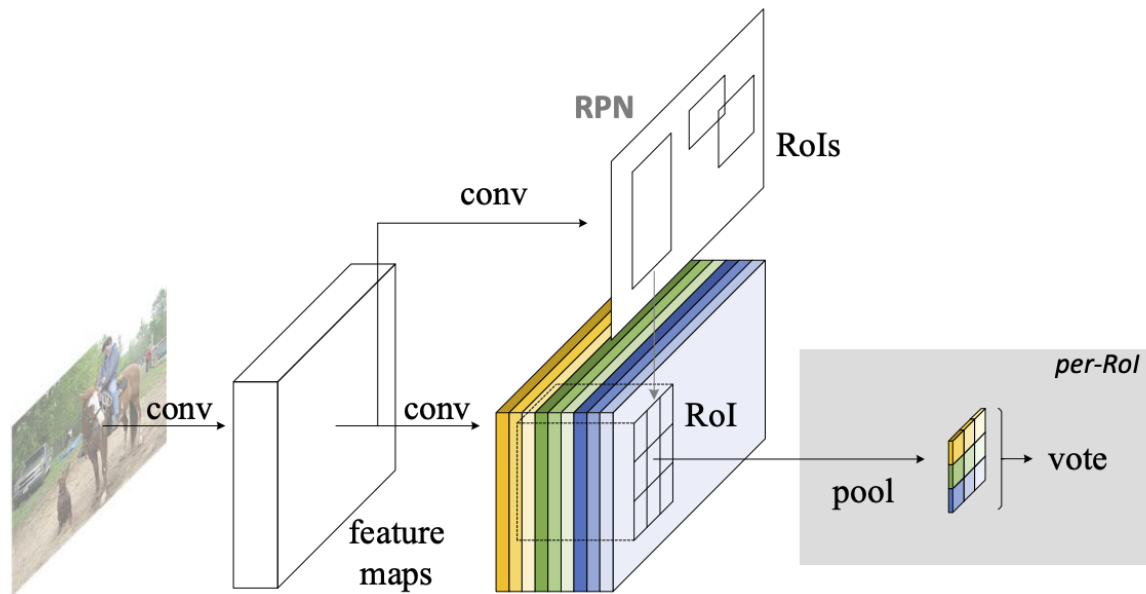
Additional Examples



Source: Lazebnik

B. Leibe, A. Leonardis, and B. Schiele, [Robust Object Detection with Interleaved Categorization and Segmentation](#), IJCV 77 (1-3), pp. 259-289, 2008.

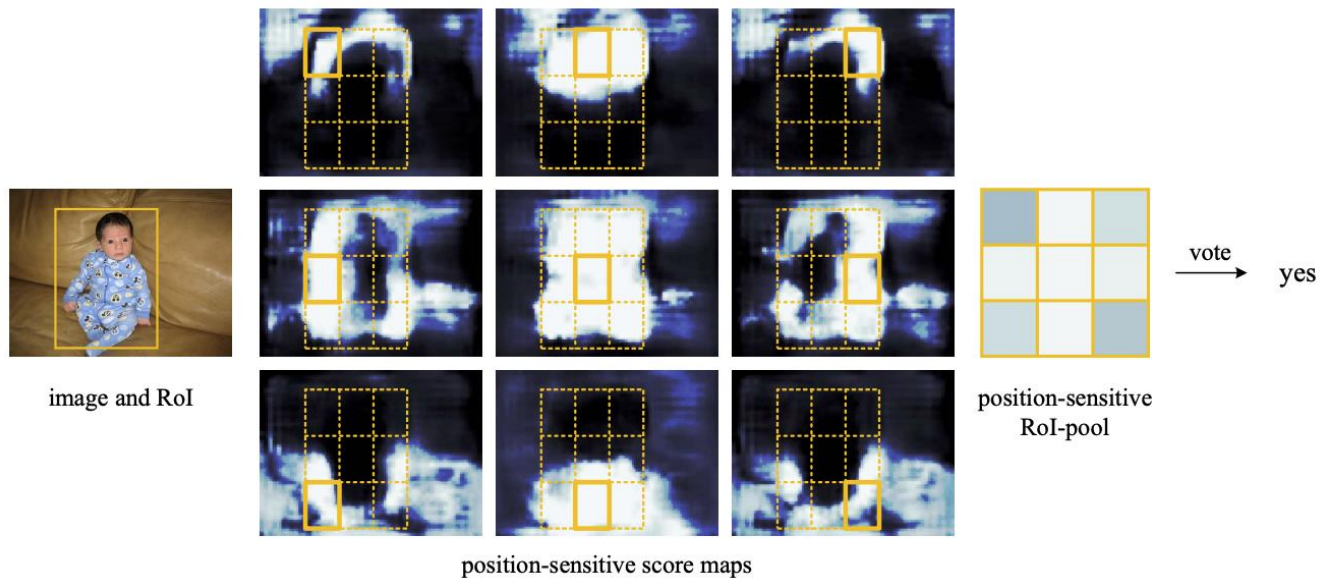
A More Recent Example: Voting for Detection



Source: Lazebnik

J. Dai et al. [R-FCN: Object Detection via Region-based Fully Convolutional Networks](#). arXiv 2016

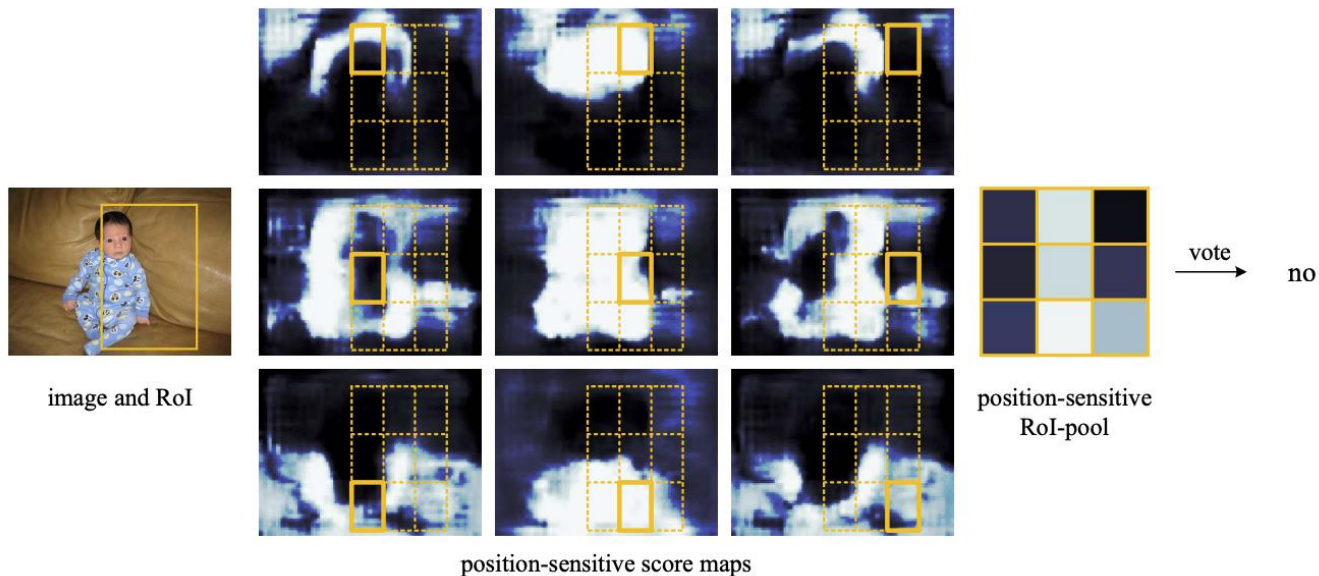
A More Recent Example: Voting for Detection



Source: Lazechnik

J. Dai et al. [R-FCN: Object Detection via Region-based Fully Convolutional Networks](#). arXiv 2016

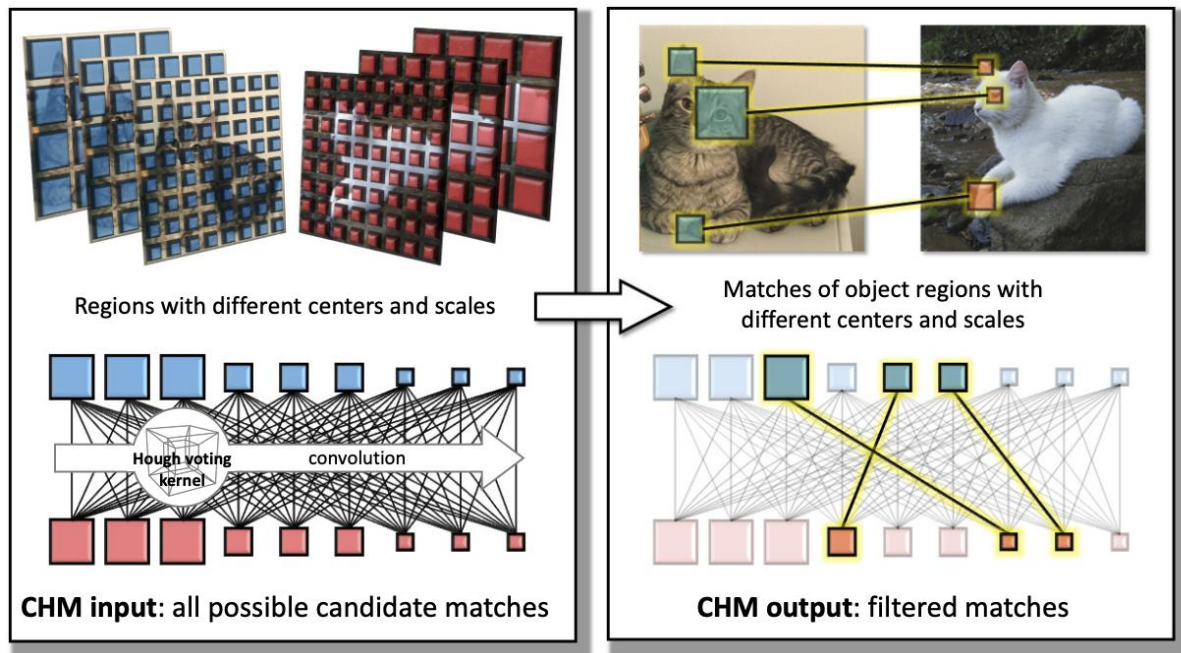
A More Recent Example: Voting for Detection



Source: Lazechnik

J. Dai et al. [R-FCN: Object Detection via Region-based Fully Convolutional Networks](#). arXiv 2016

Convolutional Hough Matching Networks



Source: Lazebnik

J. Min and M. Cho. [Convolutional Hough matching networks](#). CVPR 2021